Framework GNN-AID: Graph Neural Network Analysis Interpretation and Defense

Kirill Lukyanov[1,2,3] , **Mikhail Drobyshevskiy**[1,2] , **Georgii Sazonov**[2,3] ,
Mikhail Soloviov[2] , **Ilya Makarov**[1,4]
[1] ISP RAS Research Center for Trusted Artificial Intelligence, 109004 Moscow, Russia
[2] Ivannikov Institute for System Programming of the Russian Academy of Sciences, 109004 Moscow, Russia
[3] Moscow Institute of Physics and Technology (National Research University), 141700 Moscow, Russia
[4] Lomonosov Moscow State University, Leninskie Gory, 1, Moscow, 119991, Russia
[5] AIRI, 121170 Moscow, Russia
{lukyanov.k, drobyshevsky, sazonovg}@ispras.ru, m.solovev@phystech.edu, iamakarov@hse.ru

Abstract

The growing need for Trusted AI (TAI) highlights the importance of interpretability and robustness in machine learning models. However, many existing tools overlook graph data and rarely combine these two aspects into a single solution. Graph Neural Networks (GNNs) have become a popular approach, achieving top results across various tasks. We introduce GNN-AID (Graph Neural Network Analysis, Interpretation, and Defense), an open-source framework designed for graph data to address this gap. Built as a Python library, GNN-AID supports advanced trust methods and architectural layers, allowing users to analyze graph datasets and GNN behavior using attacks, defenses, and interpretability methods.

GNN-AID is built on PyTorch-Geometric, offering preloaded datasets, models, and support for any GNNs through customizable interfaces. It also includes a web interface with tools for graph visualization and no-code features like an interactive model builder, simplifying the exploration and analysis of GNNs. The framework also supports MLOps techniques, ensuring reproducibility and result versioning to track and revisit analyses efficiently.

GNN-AID is a flexible tool for developers and researchers. It helps developers create, analyze, and customize graph models, while also providing access to prebuilt datasets and models for quick experimentation. Researchers can use the framework to explore advanced topics on the relationship between interpretability and robustness, test defense strategies, and combine methods to protect against different types of attacks.

We also show how defenses against evasion and poisoning attacks can conflict when applied to graph data, highlighting the complex connections between defense strategies.

GNN-AID is available at github.com/ispras/GNN-AID

1 Introduction

In recent years, trustworthiness has emerged as a critical area of research in artificial intelligence. It encompasses key aspects such as interpretability and security. Numerous methods for interpreting machine learning models have been proposed for various data domains, including images, text, and tabular data [Burkart and Huber, 2021]. At the same time, new attack strategies targeting machine learning models, as well as corresponding defense mechanisms, are continually being developed [Tian *et al.*, 2022].

Access to diverse tools within a unified interface is highly advantageous for a machine learning developer. This allows for the application of multiple techniques without the need to search for and integrate separate implementations. To address this, several frameworks and toolboxes have been introduced that combine model interpretability, attack methods, defense mechanisms, and other functionalities. However, these frameworks often pay little attention to the graph data domain despite the rapid growth and active development of GNNs in recent years.

To fill this gap, we present GNN-AID, a tool fully dedicated to Graph Neural Networks. GNN-AID provides a comprehensive set of tools for analyzing, interpreting, attacking, and defending GNN models. It includes an extensible Python library complemented by a frontend module that offers most of the functionality through an intuitive interface. In this paper, we provide the following. Section 2 gives a background and reviews alternative tools, in section 3 we describe the system architecture and its key components. Section 4 outlines the implemented tools for interpretation, attacks, and defenses, and section 5 presents the main usage scenarios for the framework, followed by a conclusion section 7 discussing future directions.

2 Background

In this section, we give a background on GNN models and methods for interpretation, attack, and defense and overview existing tools and libraries.

2.1 Graph neural networks and trusted AI methods

Graph Neural Networks started to gain significant attention in 2016, when the first convolutional layers — GCN [Kipf and Welling, 2016], SAGE [Hamilton *et al.*, 2017], GAT [Veličković *et al.*, 2017], and GIN [Xu *et al.*, 2018] later — were introduced. These architectures demonstrated state-of-the-art (SOTA) performance across various tasks, making them the most popular choices for graph-based problems. Typical tasks in the graph domain include node classification, link prediction, graph classification, graph generation, graph similarity measurement, and others. Usually, GNN architectures consist of convolutional layers that learn vector representations of a graph and/or its nodes, followed by a classifier that solves the target task. The specificity of GNNs from the models working with other data types lies in these convolutional layers.

Interpretation methods for machine learning models can be categorized into *post-hoc* methods, which provide explanations after model training, and *self-interpretable* methods, where explanations are built-in and generated during the training and included as part of the model's output. For example, in node classification tasks, a local post-hoc explanation method would identify important graph elements (e.g., specific nodes or edges) within the neighborhood of a given node that mostly influenced the model's prediction. Self-interpretable models often provide a broader range of explanation types. For instance, NeuronAnalysis [Han *et al.*, 2022] in global interpretation can associate logical expressions with neurons in the final layer to describe their role in the model. However, there are very few such methods in the literature. Another approach is counterfactual interpretation, which involves an expert interacting with a model by posing "what-if" questions and observing changes in its behavior. While these techniques are promising, GNN-AID does not yet support them.

Attacks on machine learning models can be broadly categorized into several types: *evasion attacks* [Goodfellow *et al.*, 2014], which aim to mislead models by manipulating inputs at inference time; *poisoning attacks* [Zhang *et al.*, 2022a], where adversaries compromise the training process by introducing malicious data, including *backdoor attacks* [Zheng *et al.*, 2023], where hidden triggers are implanted to induce specific behaviors during inference; *privacy attacks* [Shaikhelislamov *et al.*, 2023], which include attempts to infer sensitive information about the training data, such as Membership Inference (MI) attacks; *model stealing attacks* [Oliynyk *et al.*, 2023], where adversaries attempt to replicate or extract a model's architecture and parameters for unauthorized use; and *interpretation attacks* [Wang and Gong, 2022], which aim to manipulate or mislead interpretability mechanisms, reducing trust in the model's explanations.

Now, the framework supports post-hoc interpretation and self-interpretable methods, both evasion and poisoning attacks, along with corresponding defense methods to enhance the robustness of GNNs. Other types of attacks, such as privacy attacks, model stealing, and interpretation attacks, are not yet supported, but architectural extensions are already in place to facilitate the addition of privacy-related attacks, such as Membership Inference (MI) attacks, which are planned for implementation shortly.

2.2 Trusted AI frameworks

From the perspective of open source tools and frameworks, existing solutions can be broadly divided into those aimed at ensuring the trustworthiness of AI models across various input data types, typically images, text, and tabular data, and those specifically focused on graph data processing.

Frameworks for trustworthiness

Several frameworks are designed to ensure the trustworthiness of AI models across various domains. There are ones focused on interpretability, another incorporates attack and defense methods. Most popular tools for explainable AI are designed for images, text and tabular data and do not support graph neural networks: AI Explainability 360 (Python) [Arya *et al.*, 2021], Google Cloud's AI Explanations [], Python ELI5[1], InterpretML [Nori *et al.*, 2019], IML [Molnar *et al.*, 2018], DeepExplain [Ancona *et al.*, 2017], omnixai [Yang *et al.*, 2022]. Limited support of GNNs is present in Microsoft AzureML-Interpret[2].

Frameworks focusing on attacks and defenses include CleverHans [Papernot *et al.*, 2018], Foolbox [Rauber *et al.*, 2017], and ART [Nicolae *et al.*, 2018]. They can support GNNs but their methods need adaptation for graph data.

Graph-specific libraries

Now we consider libraries specifically developed for GNN analysis that include some TAI functionality. Table 1 gives a comparison. Probably, the most popular one is PyTorch Geometric (PyG) [Fey and Lenssen, 2019]: a widely-used library for building GNNs, offering datasets and prebuilt models. It provides several post-hoc local interpretation algorithms. Deep Graph Library (DGL) [Wang *et al.*, 2019]: A flexible and scalable GNN framework that supports multiple backends including PyTorch. It implements only one interpretation method. DIG [Liu *et al.*, 2021]: A comprehensive library focusing on various GNNs tasks and includes an explainability tools package with a more diverse set of algorithms. DeepRobust [Li *et al.*, 2020] and GreatX [Li *et al.*, 2022] are focused on attacks and defenses on graph data and provide a bunch of methods. GraphFramEx [Amara *et al.*, 2022] and CLARUS [Metsch *et al.*, 2024] are designed specifically for GNN interpretability. While CLARUS offers certain interpretation methods and has a web page with interactive design, its codebase has not been maintained since 2022.

While there are many tools available, none provides a comprehensive set of functionalities for interpretation, attacks, and defenses tailored specifically for graph data. Moreover, very few tools include a user interface, which significantly

[1] https://github.com/eli5-org/eli5

[2] https://docs.microsoft.com/python/api/overview/azure/ml/?view=azure-ml-py

Name	Interpretation	Attacks	Defenses	GUI
PyG	5	-	-	-
DGL	1	-	-	-
DIG	8	-	-	-
DeepRobust	-	12	13	-
GreatX	-	18	17	-
GraphFramEx	15	-	-	-
CLARUS	4	-	-	+
GNN-AID	8	7	7	+

Table 1: Comparison of tools for GNNs analysis with numbers of corresponding algorithms.

lowers the entry barrier for beginners. GNN-AID aims to address these gaps by offering an integrated, extensible framework with a focus on usability and functionality for graph-based machine learning.

3 System Overview

3.1 Architecture and pipelines

The main general framework pipeline can be seen in the diagram Figure 1 Additional information about these steps is provided below.

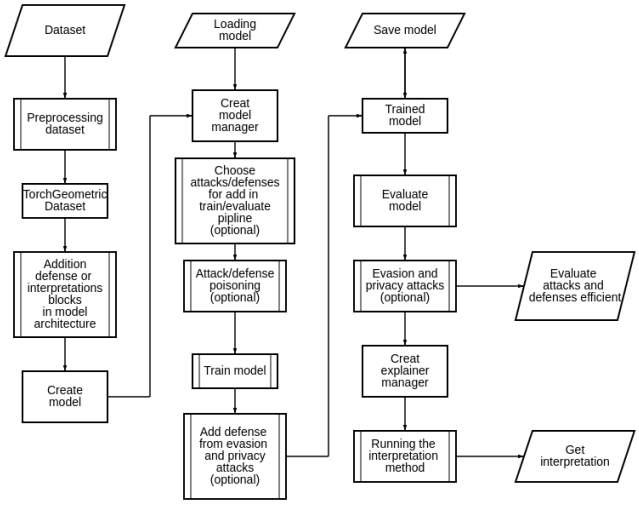

Figure 1: GNN-AID main pipleine.

Dataset processing module

The first step in using GNN-AID is dataset selection. It consists of two stages: selecting a graph or a set of graphs (for graph classification tasks) and defining the dataset's variable components: features, labels, and the target task.

This separation accounts for the flexibility of solving multiple tasks on the same graph. For instance, a graph can support different feature construction methods, labeling schemes, or task definitions, such as switching from node classification to edge prediction. Attributes (e.g., a person's age in a social graph or atom types in a molecular graph) are distinguished from features, which are numerical representations derived from these attributes.

Model, model manager, attacks, and defenses

The model is constructed as a modular system using universal building blocks that may include the following components:

- A graph convolutional layer or a fully connected layer
- A batch normalization layer
- An activation function
- A dropout layer

For node classification tasks, pooling layers must be placed between blocks to define the representations of nodes and the entire graph. In addition to sequential connections, blocks can also be linked using skip connections.

The model manager, in turn, is responsible for training the model, saving it, loading it, performing attacks on the model, and applying defense methods. The process of training, conducting attacks, and applying defenses consists of the following stages illustrated in Algorithm 1

It should be noted that, in addition to the listed actions, user-defined hooks are supported to modify the model's behavior and/or the training process.

Defenses against privacy attacks and evasion attacks are executed before and/or after training on a batch since most of these methods require data modification (for example, Adversarial Training) or gradient modification (for example, Gradient Regularization). Additionally, some methods modify the model's architecture itself; such modifications are performed once before training begins unless otherwise specified by the defense method.

Each attack and defense method can be defined or omitted independently of others, enabling complex experiments and the exploration of interactions between different defenses, which will be discussed in more detail in Section 5.

Attack and defense methods inherit from the `Defender` and `Attacker` classes. Subsequently, classes for specific types of attacks and defenses: evasion, poisoning, and privacy are derived from these base classes.

Thus, to add a new attack or defense method to the registry, it is sufficient to inherit from the appropriate attack or defense class and override the methods of the parent class as needed.

Interpretation module

The interpretation module consists of an interpretation manager, which is responsible for saving and loading interpretation results, executing interpretation methods, and calculating interpretation metrics (e.g., fidelity).

The interpretation methods themselves inherit from the `Explainer` class, which can be overridden to add a new interpretation method to the method registry. If the interpretation method falls under self-interpretable methods and requires specific architectural features of the model, it is necessary to additionally define the constraints of the interpretation method.

After training the model, the user must specify the model manager and the interpretation method to be executed.

Algorithm 1 Train and Evaluate Pipeline with Defenses and Attacks

Require: Initial dataset $\mathcal{D}$, its training part $\mathcal{D}_{train}$, dataset's part chosen by boolean mask $\mathcal{D}_{mask}$, model $\mathcal{M}$ with parameters Θ, number of epochs N, loss after train L, batch size B, early stopping criteria, optimization algorithm, all attacks and defenses methods are used if they were chosen

$\mathcal{D}_{train} \leftarrow$ PoisonAttack($\mathcal{D}_{train}$) {Applying poison attacks to the training part of the dataset}
$\mathcal{D}_{train} \leftarrow$ PoisonDefense($\mathcal{D}_{train}$) {Applying poison defense to mitigate poison attack}
Initialize $\mathcal{M}$ with parameters Θ_0 {Model initialization}
for $epoch = 1$ to N **do**
  $\mathcal{B} \leftarrow$ SplitIntoBatches(D_{train}, B)
  for each $b \in \mathcal{B}$ **do**
    $\mathcal{M}, b, b* \leftarrow$ PrivacyDefense($\mathcal{M}, b$) {Add defense against privacy attacks before training on batch}
    $\mathcal{M}, b, b* \leftarrow$ EvasionDefense($\mathcal{M}, b, b*$) {Add defense against evasion attacks before train on batch}
    $\mathcal{M}, \mathcal{L} \leftarrow$ Train on batch($\mathcal{M}, b$) {Train model on the current batch}
    $\mathcal{M}, \mathcal{L} \leftarrow$ PrivacyDefense($\mathcal{M}, \mathcal{L}$) {Add defense against privacy attacks after training on batch}
    $\mathcal{M}, \mathcal{L} \leftarrow$ EvasionDefense($\mathcal{M}, \mathcal{L}$) {Add defense against evasion attacks after training on batch}
    $\Theta \leftarrow$ OptimizationStep($\mathcal{M}, \mathcal{L}, \Theta$)
    if EarlyStoppingCriteriaMet() **then**
      break
    end if
  end for
end for
$\mathcal{D}_{mask} \leftarrow$ EvasionAttacks($\mathcal{M}, \mathcal{D}_{mask}$) {Applying evasion attack}
$\mathcal{M} \leftarrow$ PrivacyAttacks($\mathcal{M}, \mathcal{D}_{mask}$) {Applying privacy attack}
$Metrics \leftarrow$ EvaluateModel($\mathcal{M}, \mathcal{D}_{mask}$) {Evaluate model on $\mathcal{D}_{mask}$}
return $\mathcal{M}$, $Metrics$

3.2 Frontend

The web interface of GNN-AID replicates most backend capabilities, allowing users to interact with the tool through a graphical UI in their browser. The standard workflow includes selecting or building a dataset, constructing or loading a GNN model, and training the model. After that, a user can go three ways.

1. Analyzing graph data. This scenario enables the visualization of the training process. Users can observe detailed intermediate outputs, such as weight matrices from each layer, logits and predictions for each node, and the current performance metrics on training, validation, and test sets.

2. Interpreting GNN models. In the case of post-hoc interpretation, an interpretation algorithm is specified after model training. It provides either a local or global interpretation. Alternatively, the user can build a self-interpretable model and receive interpretation during the training process.

3. Applying attacks and defenses to GNN models. Similar to the previous scenario, the user specifies the parameters for attacks and/or defenses.

Figure 2 shows a screenshot for example.

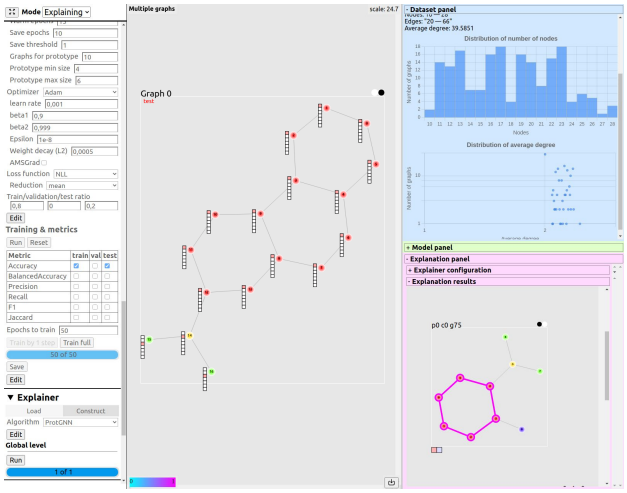

Figure 2: Screenshot of the web interface in interpretation mode. The left side is a menu panel for selecting the dataset (not visible), model training parameter, etc. (additional elements are visible after scrolling down). In the center, there is a visualization of a molecule graph from the MUTAG dataset with additional information around the nodes. The right panel contains some dataset statistics (the upper part), and the global model interpretation via the ProtGNN method. A prototype is shown as a purple highlighted subgraph.

3.3 Summary

The modular architecture enables rapid expansion of the framework's supported methods for attacks, defenses, and interpretations. While the interaction between modules is based on PyTorch, all modules rely on core methods that can be overridden by the user. Through an additional interface layer, the framework can be adapted to work with models and methods built on other libraries or even other programming languages, provided the core methods are correctly implemented.

The availability of a web interface allows users with limited expertise in graph-based machine learning methods to interact with the framework effectively. Additionally, the interface provides visualization for each stage of interaction with the framework.

4 Implemented Methods in the Framework

Here we describe interpretation methods, attacks, and defense algorithms implemented in GNN-AID.

4.1 Interpretation methods

GNNExplainer

GNNExplainer [Ying *et al.*, 2019] identifies a subgraph that maximizes the mutual information between the model's predictions and distribution of possible subgraph structures by training a differentiable mask over edges.

PGExplainer
PGExplainer [Luo *et al.*, 2020] uses a parametric model to identify influential edges and nodes by learning probabilities of their contribution to predictions. It employs thresholding on predicted edge probabilities to highlight the most significant graph components.

PGMExplainer
PGMExplainer [Vu and Thai, 2020] constructs a probabilistic model to determine important subgraphs influencing specific node predictions. By minimizing the prediction discrepancy between the full graph and subgraphs, it evaluates node and edge importance.

SubgraphX
SubgraphX [Yuan *et al.*, 2021] uses the Shapley value and Monte Carlo Tree Search (MCTS) to generate a connected subgraph that best explains the model's predictions. It evaluates subgraph importance based on contribution to model output.

ZORRO
ZORRO [Funke *et al.*, 2020] selects important nodes and features from a computational subgraph by masking components and injecting noise. The algorithm greedily iterates through the nodes and features of the original computation graph and produces a minimal set S that would correspond to the set value of the $fidelity$ metric.

GraphMask
The main goal of GraphMask [Schlichtkrull *et al.*, 2020] is to identify edges in a graph that can be removed (considered "redundant") without affecting the model's predictions. The algorithm uses stochastic binary switches $z_{u,v}$ to determine whether to keep the edge (u, v) or replace it with a baseline value on layer k:

$$\hat{m}_{u,v}^{(k)} = z_{u,v}^{(k)} \cdot m_{u,v}^{(k)} + (1 - z_{u,v}^{(k)}) \cdot b^{(k)}$$

Each value $z_{u,v}^{(k)}$ is determined by a learned function $g_\pi(h_u^{(k-1)}, h_v^{(k-1)}, m_{u,v}^{(k)})$, which is implemented as a neural network. Here $h_u^{(k-1)}$, $h_v^{(k-1)}$ are the hidden states of the nodes, and $m_{u,v}^{(k)}$ is the original message. g_π is trained on a training data set, which allows it to identify common patterns in graphs and work not only on the training set but also on new (test) data.

ProtGNN
ProtGNN [Zhang *et al.*, 2022b] uses the paradigm of prototype learning to build self-interpretable GNN. This method implies the attachment of the prototype layer to any GNN backbone that stores embeddings of prototype graphs for each predicted class. Firstly, the backbone produces embedding for the input graph, then it is compared with each prototype, and similarity metrics are passed to the linear classifier for producing the output label. As the prototype layer stores just a trainable tensor, for interoperability, each embedding within it is projected into one of the graphs from the train set. This projection is performed by detecting a subgraph that has the most similar embedding to the prototype embedding. Monte-Carlo Tree Search (MCTS) is used for subgraph definition.

NeuronAnalysis
NeuronAnalysis [Han *et al.*, 2022] is a post-hoc interpretation method for graph neural networks that focuses on the explanation of distinct neuron behavior within GNN. An algorithm adopts a concept-learning technique of interpretation when a human understanding of logical concepts serves as an explanation. Each concept is a combination of basic concepts with logic operations: NOT, AND, OR. Basic ones can be domain-based (for molecular datasets like MUTAG [Debnath *et al.*, 1991] it can be "this node is a carbon atom") or general concepts for every graph that specifies graph structure (for example, "this node has degree > 5").

4.2 Attack methods

RandomPoison
The RandomPoison is a poisoning attack that randomly removes a specified percentage of edges from a graph to degrade the model's performance.

Nettack
Nettack [Zügner *et al.*, 2018] modifies graph structure and features through a bi-level optimization process: the first level maximizes the impact on the classification of the target node, while the second level minimizes the changes to the graph to ensure they remain as inconspicuous as possible. The attack involves two types of modifications: alterations to the graph structure (such as adding or removing edges) and modifications to the node features (adjusting feature values).

Fast Gradient Sign Method (FGSM)
FGSM [Goodfellow *et al.*, 2014] is a white-box evasion attack method that follows the idea of changing input in the opposite direction of the gradient of loss function:

$$x^{'} = x + \epsilon \, sign(\nabla l(f(x), y))$$

Projected Gradient Descent (PGD)
PGD [Madry *et al.*, 2017] is an iterative modification of the FGSM white-box evasion attack method. The main difference is that the data shift is done in several steps. After each step, the gradient signs are recalculated:

$$x_{i+1}^{'} = \text{Clip}_\epsilon \left\{ x_i^{'} + \alpha \cdot \text{sign} \left[\nabla l(f(x), y) \right] \right\}$$

where $x_{i+1}^{'}$ denotes the changed input data since the previous iteration, Clip$\{\}$ limits the resulting data shift to no more than ϵ and α denotes the shift step size at each iteration.
In addition to the attack on graph features, the GNN-AID PGD supports an attack on the graph structure. In this case, the goal of the PGD [Xu *et al.*, 2019] attack is to change the graph structure to maximize the classification error of the target node. After solving the optimization problem using gradient descent, a random sampling method is used to generate discrete perturbations, since changes in the graph must be discrete (e.g. adding or removing edges).

Q-Attack
Q-Attack [Chen *et al.*, 2019] is designed especially for community detection tasks and removes edges that connect nodes with its communities and adds edges to other ones. This pair

of operations (remove/add one edge) is called rewiring and is considered a preferable way of attacking edges because it does not change graph statistics such as several edges or degree distribution.

As the method was created for community detection tasks, the fitness function of the genetic algorithm uses modularity of graph, so it requires community labeling. However, in other tasks, it is also possible to use labels that being predicted by GNN. For generalization purposes, GNN-AID uses these predicted answers as labels for modularity calculation and fitness function.

Contrastive Loss Gradient Attack (CLGA)

CLGA [Zhang *et al.*, 2022a] is a graph poison attack method that works in an unsupervised way. Within each iteration, the algorithm produces two augmented views of the graph and treats different views of one node as a positive pair and views of different nodes as a negative pair. Contrastive loss is calculated on these two views and edge to addition/deletion is chosen based on the gradient sign and corresponding value of adjacency matrix.

MetaAttack

MetaAttack [Zügner and Günnemann, 2019] is a poison attack method based on meta-learning. It considers graph matrix structure as a hyperparameter and calculates gradients concerning it. Therefore adjacency matrix is modified edge-by-edge on each iteration of the algorithm. MetaAttack works in the black-box scenario and trains a surrogate model on each iteration for obtaining unknown labels. Computation of meta-gradients may be expensive and authors propose a variant of method with approximate calculation of them. GNN-AID supports both options (MetaAttackFull and MetaAttackApprox respectively).

4.3 Defense methods

Gradient Regularization

Modifying of loss function used by a model with additional gradient regularization [Finlay and Oberman, 2021] can serve as a defense method too.

$$L = l(f(x), y) + \lambda(\frac{1}{h^2 n} \left\| f(z) - f(x) \right\|_2^2),$$

where

$$z = x + h \, \frac{\nabla l(f(x), y)}{\left\| \nabla l(f(x), y) \right\|_2},$$

h is a quantization step and λ is a regularization coefficient.

Defensive Distillation

Distillation as a defense method is about creating a copy of the original model that is more robust to attacks. The new model uses the original one as a teacher and so-called smooth labels for this model are obtained using the $softmax(x, T)$ on the last layer of the teacher:

$$softmax(x, T)_i = \frac{e^{x_i/T}}{\sum_j e^{x_j/T}}$$

Adversarial Training

Adversarial training [Goodfellow *et al.*, 2014] is a defense technique that implies adding adversarial examples in a train set. Therefore adversarial part is added to the loss function:

$$L = l(f(x), y) + \lambda \, l(f(x^{'}, y)),$$

where $x^{'}$ is adversarial sample and λ is adversarial training coefficient.

Data Quantization Defense

Quantization is a preprocessing technique that transforms continuous values into a discrete set of values arranged on a uniform grid [Guo *et al.*, 2017]. While this method may diminish the quality of the original data, it can effectively mitigate the effects of adversarial attacks. Consequently, the fault diagnosis model needs to be retrained using the quantized data.

Autoencoder Defense

Autoencoders can be used to perform robust training too as it was shown in [Meng and Chen, 2017]. In this case, minimized loss function:

$$L = \left\| x_{AE} - x \right\|_1,$$

where $x_{AE} = autoencoder(x + \epsilon)$ is a reconstructed data and ϵ is added noise.

JaccardDefense

JaccardDefense [Wu *et al.*, 2019] is a poison defender that removes edges between nodes that are not similar concerning the Jaccard Index (binary features of nodes being compared). This is followed by the idea that many attack methods are trying to connect not similar nodes to shadow important links.

GNNGuard

GNNGuard [Zhang and Zitnik, 2020] modifies message-passing functions to diminish the influence of suspicious edges. This is achieved with additional defense weights that are multiplied with passed messages and with node embedding from previous layers. These defense weights are jointly learned with network parameters.

5 Use Cases

5.1 Educational use case

The GNN-AID framework can be used as an educational tool for beginning GNN learners. It provides real-time visualization of the training process on small demonstration graphs. Users can observe the model's trainable weights; node features, embeddings, predictions, and true labels directly on the graph. Additionally, the system generates plots of various training metrics, offering deeper insights into the learning dynamics. Screenshots can be found in the appendix.

5.2 Developer use case

The GNN-AID framework can be used to design, test, and experiment with GNN models. It offers modular APIs for custom architectures and datasets, and built-in tools for interpretability, robustness evaluation, and attack/defense analysis. The framework also provides visualization of node embeddings, predictions, and graph-level statistics, streamlining workflows.

5.3 Researcher use case

The GNN-AID framework can be used to study the interactions between various attack and defense types in GNNs. It enables researchers to explore the relationship between model interpretability and robustness. The framework includes MLOps tools for ensuring reproducibility and versioning of results. Additionally, it can be used to create benchmarks for comparing interpretation methods, attack strategies, and defense mechanisms in graph-based models.

6 Conflicting interactions among protection mechanisms

As demonstrated in [Szyller and Asokan, 2023], defense methods can conflict in the image domain. In this work, we further investigate whether such conflicts are characteristic of defense methods in the graph domain. Unlike images, attacks and defenses on graphs can modify not only features but also the graph structure.

For the experiment, the following defense methods were selected: JaccardDefense (Jaccard), Adversarial Training (AdvTrain), and Gradient Regularization (GradGeg). The first method is designed to defend against poisoning attacks, while the other two target evasion attacks. The attacks considered were PGD as an evasion attack and CLGA as a poisoning attack. The experiments were conducted on the Cora dataset. Detailed information on the hyperparameters of the attack and defense methods is provided in the supplementary materials.

According to the training pipeline described in Algorithm 1, with the specified attack and defense methods, the poisoning attack was applied first, followed by the poisoning defense method. During training, a defense against evasion attacks was added, and before model evaluation, an evasion attack was applied to the test data.

The experiment examined various combinations of attack and defense methods of both types. Cases where a defense against one type of attack was added while the only attack was of the other type were not considered.

The results of the experiments were averaged over 15 runs.

—	No EvDefense	AdvTrain	GradReg
No PoisDefense	90 ± 0	88 ± 2	91 ± 1
Jaccard	82 ± 5	81 ± 5	80 ± 5

Table 2: Accuracy of a model with different combinations of defense methods when not under attack on Cora dataset

—	No EvDefense	AdvTrain	GradReg
No PoisDefense	57 ± 3	78 ± 3	70 ± 4
Jaccard	46 ± 4	69 ± 5	63 ± 2

Table 3: Accuracy of a model with different combinations of defense methods when model attack only by evasion atttack PGD on Cora dataset

—	No EvDefense	AdvTrain	GradReg
No PoisDefense	67 ± 2	—	—
Jaccard	76 ± 6	67 ± 3	61 ± 2

Table 4: Accuracy of a model with different combinations of defense methods when model attack only by poison attack CLGA on Cora dataset

—	No EvDefense	AdvTrain	GradReg
No PoisDefense	—	—	—
Jaccard	—	57 ± 4	42 ± 1

Table 5: Accuracy of a model with different combinations of defense methods when model attack by PGD and CLGA together on Cora dataset

From Tables 3 and 4, it is evident that defense methods against evasion attacks significantly improved the model's performance under PGD attacks. Similarly, the defense method against poisoning attacks effectively mitigated the negative impact of the CLGA attack.

However, as shown in Tables 2, 3, 4, and 5, combining defense methods degraded the overall accuracy metric in all cases. Furthermore, in each pair, adding a defense method unrelated to the type of attack being conducted worsened the model's performance. In scenarios involving combined attacks with both methods, none of the defense combinations effectively countered the combined threat.

This experiment summarizes that the graph domain is also susceptible to conflicts between defense mechanisms, similar to the image domain. Additionally, this highlights the importance of studying not only individual trustworthiness criteria but also the ability to ensure their compatibility. Without such studies, the deployment of AI systems in critical infrastructure will remain highly uncertain.

7 Conclusion and Future Work

We created a framework, GNN-AID, that provides methods for analyzing, interpreting, and defending Graph Neural Networks (GNNs) while addressing key aspects of Trusted AI: interpretability and robustness. By combining ease of use, access to no-code solutions, API, and visualization tools with MLOps support for reproducibility, GNN-AID enables new opportunities for researchers and developers alike.

In this article, we used GNN-AID to demonstrate for the first time the intricate relationship between defense strategies against evasion and poisoning attacks in the context of graph data. This finding underscores the complexity of designing effective defenses and highlights the need for a holistic approach to building trusted GNN and other AI models.

GNN-AID is a versatile tool that fosters scientific exploration and industrial solutions for graph data analysis, advancing the principles of Trusted AI.

The main direction for future work is to expand the range of available attacks, defenses, and interpretation methods. Additionally, this includes introducing a class of privacy attacks and corresponding defense mechanisms.

References

[Amara *et al.*, 2022] Kenza Amara, Rex Ying, Zitao Zhang, Zhihao Han, Yinan Shan, Ulrik Brandes, Sebastian Schemm, and Ce Zhang. Graphframex: Towards systematic evaluation of explainability methods for graph neural networks. *arXiv preprint arXiv:2206.09677*, 2022.

[Ancona *et al.*, 2017] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. *arXiv preprint arXiv:1711.06104*, 2017.

[Arya *et al.*, 2021] Vijay Arya, Rachel KE Bellamy, Pin-Yu Chen, Amit Dhurandhar, Michael Hind, Samuel C Hoffman, Stephanie Houde, Q Vera Liao, Ronny Luss, Aleksandra Mojsilović, et al. Ai explainability 360 toolkit. In *Proceedings of the 3rd ACM India Joint International Conference on Data Science & Management of Data (8th ACM IKDD CODS & 26th COMAD)*, pages 376–379, 2021.

[Burkart and Huber, 2021] Nadia Burkart and Marco F Huber. A survey on the explainability of supervised machine learning. *Journal of Artificial Intelligence Research*, 70:245–317, 2021.

[Chen *et al.*, 2019] Jinyin Chen, Lihong Chen, Yixian Chen, Minghao Zhao, Shanqing Yu, Qi Xuan, and Xiaoniu Yang. Ga-based q-attack on community detection. *IEEE Transactions on Computational Social Systems*, 6(3):491–503, 2019.

[Debnath *et al.*, 1991] Asim Kumar Debnath, Rosa L Lopez de Compadre, Gargi Debnath, Alan J Shusterman, and Corwin Hansch. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. correlation with molecular orbital energies and hydrophobicity. *Journal of medicinal chemistry*, 34(2):786–797, 1991.

[Fey and Lenssen, 2019] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

[Finlay and Oberman, 2021] Chris Finlay and Adam M Oberman. Scaleable input gradient regularization for adversarial robustness. *Machine Learning with Applications*, 3:100017, 2021.

[Funke *et al.*, 2020] Thorben Funke, Megha Khosla, and Avishek Anand. Hard masking for explaining graph neural networks. 2020.

[Goodfellow *et al.*, 2014] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[Guo *et al.*, 2017] Chuan Guo, Mayank Rana, Moustapha Cisse, and Laurens Van Der Maaten. Countering adversarial images using input transformations. *arXiv preprint arXiv:1711.00117*, 2017.

[Hamilton *et al.*, 2017] L. Hamilton, W., R. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Proceedings of International Conference on NIPS*, pages 1025–1035, Red Hook, NY, USA, 2017. Curran Associates Inc.

[Han *et al.*, 2022] Xuanyuan Han, Pietro Barbiero, Dobrik Georgiev, Lucie Charlotte Magister, and Pietro Li'o. Global concept-based interpretability for graph neural networks via neuron analysis. In *AAAI Conference on Artificial Intelligence*, 2022.

[Kipf and Welling, 2016] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[Li *et al.*, 2020] Yaxin Li, Wei Jin, Han Xu, and Jiliang Tang. Deeprobust: A pytorch library for adversarial attacks and defenses. *arXiv preprint arXiv:2005.06149*, 2020.

[Li *et al.*, 2022] Jintang Li, Bingzhe Wu, Chengbin Hou, Guoji Fu, Yatao Bian, Liang Chen, Junzhou Huang, and Zibin Zheng. Recent advances in reliable deep graph learning: Inherent noise, distribution shift, and adversarial attack. *arXiv preprint arXiv:2202.07114*, 2022.

[Liu *et al.*, 2021] Meng Liu, Youzhi Luo, Limei Wang, Yaochen Xie, Hao Yuan, Shurui Gui, Haiyang Yu, Zhao Xu, Jingtun Zhang, Yi Liu, Keqiang Yan, Haoran Liu, Cong Fu, Bora M Oztekin, Xuan Zhang, and Shuiwang Ji. DIG: A turnkey library for diving into graph deep learning research. *Journal of Machine Learning Research*, 22(240):1–9, 2021.

[Luo *et al.*, 2020] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. *Advances in neural information processing systems*, 33:19620–19631, 2020.

[Madry *et al.*, 2017] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *stat*, 1050(9), 2017.

[Meng and Chen, 2017] Dongyu Meng and Hao Chen. Magnet: a two-pronged defense against adversarial examples. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pages 135–147, 2017.

[Metsch *et al.*, 2024] Jacqueline Michelle Metsch, Anna Saranti, Alessa Angerschmid, Bastian Pfeifer, Vanessa Klemt, Andreas Holzinger, and Anne-Christin Hauschild. Clarus: An interactive explainable ai platform for manual counterfactuals in graph neural networks. *Journal of Biomedical Informatics*, 150:104600, 2024.

[Molnar *et al.*, 2018] Christoph Molnar, Giuseppe Casalicchio, and Bernd Bischl. iml: An r package for interpretable machine learning. *Journal of Open Source Software*, 3(26):786, 2018.

[Nicolae *et al.*, 2018] Maria-Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, et al. Adversarial robustness toolbox v1. 0.0. *arXiv preprint arXiv:1807.01069*, 2018.

[Nori *et al.*, 2019] Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. Interpretml: A unified framework for machine learning interpretability. *arXiv preprint arXiv:1909.09223*, 2019.

[Oliynyk *et al.*, 2023] Daryna Oliynyk, Rudolf Mayer, and Andreas Rauber. I know what you trained last summer: A survey on stealing machine learning models and defences. *ACM Computing Surveys*, 55(14s):1–41, 2023.

[Papernot *et al.*, 2018] Nicolas Papernot, Amir Rahmati, Ian Brown, Nicholas Carlini, Jan Hendrik Metzen, and Patrick McDaniel. Cleverhans v2.1.0: An adversarial machine learning library. *arXiv preprint arXiv:1610.00768*, 2018.

[Rauber *et al.*, 2017] Jonas Rauber, Wieland Brendel, and Matthias Bethge. Foolbox: A python toolbox to benchmark the robustness of machine learning models. *arXiv preprint arXiv:1707.04131*, 2017.

[Schlichtkrull *et al.*, 2020] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. Interpreting graph neural networks for nlp with differentiable edge masking. *arXiv preprint arXiv:2010.00577*, 2020.

[Shaikhelislamov *et al.*, 2023] Danil S Shaikhelislamov, Kirill S Lukyanov, Nikita Nikolaevich Severin, Mikhail Dmitrievich Drobyshevskiy, Ilya A Makarov, and Denis Yur'evich Turdakov. A study of graph neural networks for link prediction on vulnerability to membership attacks. *Zapiski nauchnykh seminarov POMI*, 530(0):113–127, 2023.

[Szyller and Asokan, 2023] Sebastian Szyller and N Asokan. Conflicting interactions among protection mechanisms for machine learning models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 15179–15187, 2023.

[Tian *et al.*, 2022] Zhiyi Tian, Lei Cui, Jie Liang, and Shui Yu. A comprehensive survey on poisoning attacks and countermeasures in machine learning. *ACM Computing Surveys*, 55(8):1–35, 2022.

[Veličković *et al.*, 2017] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks, 2017.

[Vu and Thai, 2020] Minh Vu and My T Thai. Pgm-explainer: Probabilistic graphical model explanations for graph neural networks. *Advances in neural information processing systems*, 33:12225–12235, 2020.

[Wang and Gong, 2022] Shen Wang and Yuxin Gong. Adversarial example detection based on saliency map features. *Applied Intelligence*, 52(6):6262–6275, 2022.

[Wang *et al.*, 2019] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.

[Wu *et al.*, 2019] Huijun Wu, Chen Wang, Yuriy Tyshetskiy, Andrew Docherty, Kai Lu, and Liming Zhu. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*, 2019.

[Xu *et al.*, 2018] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.

[Xu *et al.*, 2019] Kaidi Xu, Hongge Chen, Sijia Liu, Pin-Yu Chen, Tsui-Wei Weng, Mingyi Hong, and Xue Lin. Topology attack and defense for graph neural networks: An optimization perspective. *arXiv preprint arXiv:1906.04214*, 2019.

[Yang *et al.*, 2022] Wenzhuo Yang, Hung Le, Silvio Savarese, and Steven Hoi. Omnixai: A library for explainable ai. 2022.

[Ying *et al.*, 2019] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems*, 32, 2019.

[Yuan *et al.*, 2021] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. On explainability of graph neural networks via subgraph explorations. In *International Conference on Machine Learning*, pages 12241–12252. PMLR, 2021.

[Zhang and Zitnik, 2020] Xiang Zhang and Marinka Zitnik. Gnnguard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33:9263–9275, 2020.

[Zhang *et al.*, 2022a] Sixiao Zhang, Hongxu Chen, Xiangguo Sun, Yicong Li, and Guandong Xu. Unsupervised graph poisoning attack via contrastive loss back-propagation. In *Proceedings of the ACM Web Conference 2022*, pages 1322–1330, 2022.

[Zhang *et al.*, 2022b] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Cheekong Lee. Protgnn: Towards self-explaining graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9127–9135, 2022.

[Zheng *et al.*, 2023] Haibin Zheng, Haiyang Xiong, Jinyin Chen, Haonan Ma, and Guohan Huang. Motif-backdoor: Rethinking the backdoor attack on graph neural networks via motifs. *IEEE Transactions on Computational Social Systems*, 11(2):2479–2493, 2023.

[Zügner and Günnemann, 2019] Daniel Zügner and Stephan Günnemann. Adversarial attacks on graph neural networks via meta learning. *ArXiv*, abs/1902.08412, 2019.

[Zügner *et al.*, 2018] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2847–2856, 2018.

Appendix

1 Frontend examples

Here we provide several screenshots of UI demostrating frontend work. Figure 1 shows dataset statistics exploration on Cora citation graph[1]. Figure 2 shows no-code model constructor. Figure 3 shows educational example. Figures 4 and 5 show interpretation examples.

2 Conflicting interactions among protection mechanisms; Experimental parameters

Experiment parameters, including training parameters, model architecture, and hyperparameters of attack and defense algorithms.

The two-layer GCN model (GCN-2l) and the two-layer GIN model (GIN-2l) were trained as models.

```
GCN-2l(
  Sequential(
    (0): GCNConv(input_size, 16)
    (1): ReLU(inplace)
    (2): GCNConv(16, output_size)
    (3): LogSoftmax(inplace)
  )
)
```

Cora dataset parameters: 2708 nodes, 10556 edges, 7 classes, 1433 features.

Hyperparameters of attack and defense methods are presented in Table 1

Training parameters: Adam optimizer with default parameters and NLLLoss error function from PyTorch library were used for training, no batch splitting was used. The number of training epochs was 200 to achieve high classification accuracy on all datasets. The data was split in a ratio of 80 to 20 for training and testing, respectively.

The main metric for evaluating the models was the Accuracy metric on the test data after applying all declared attack and defense methods or on the original data if no attack or defense was declared.

Method	**Hyperparameters**
CLGA	learning_rate = 0.01 num_hidden = 256 num_proj_hidden = 32 activation = $prelu$ drop_edge_rate_1 = 0.3 drop_edge_rate_2 = 0.4 tau = 0.4 num_epochs = 3000 weight_decay = $1e-5$ drop_scheme = $degree$
PDG	$\epsilon = 0.005$ $num\_iterations = 10$ $\alpha(learning_rate) = 0.0005$
Jaccard	threshold = 0.4
Adversarial training	attack_name = $FGSM$ $\epsilon = 0.01$
Gradient Regularization	regularization_strength = 50

Table 1: Hyperparameters of attack and defense methods

[1] https://paperswithcode.com/dataset/cora

[2] https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.BAShapes.html

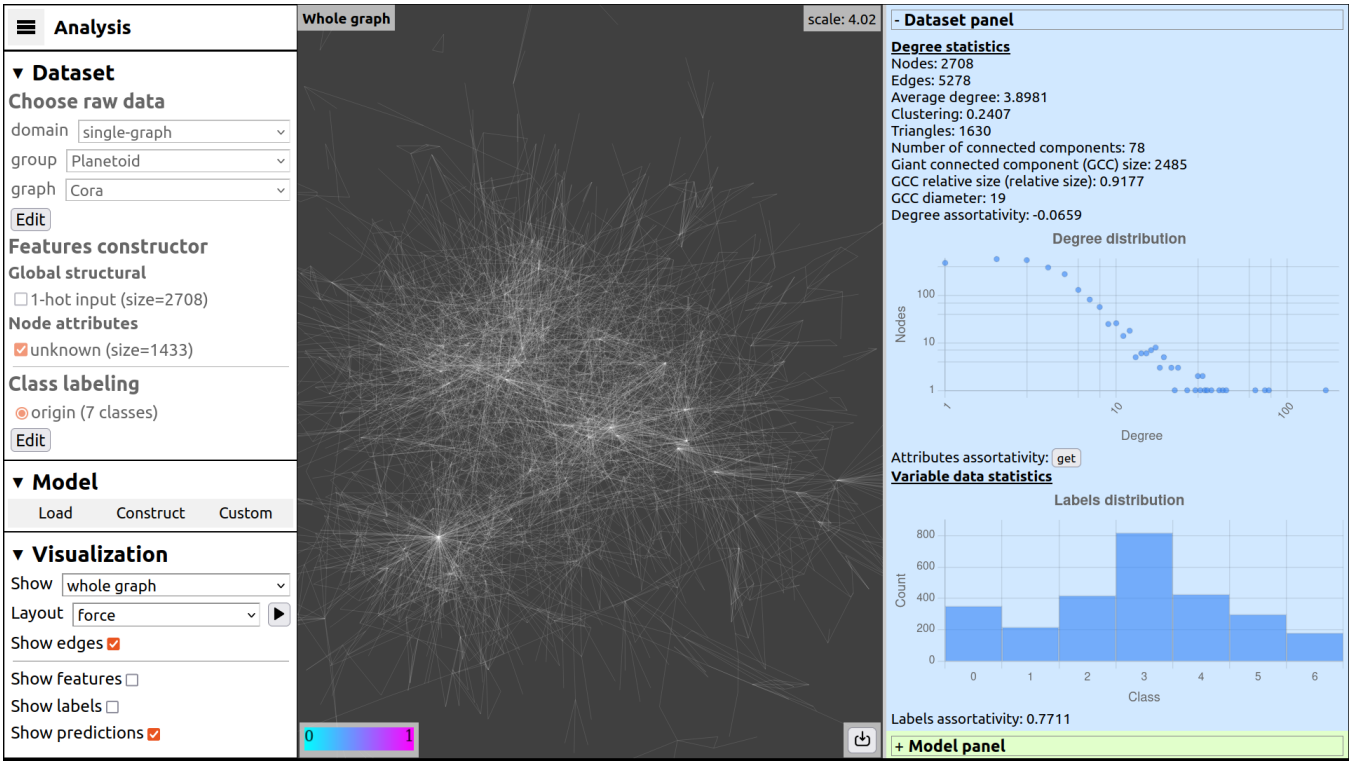

Figure 1: Screenshot of dataset statistics for Cora citation graph. Graph is zoomed out so its nodes are not shown.

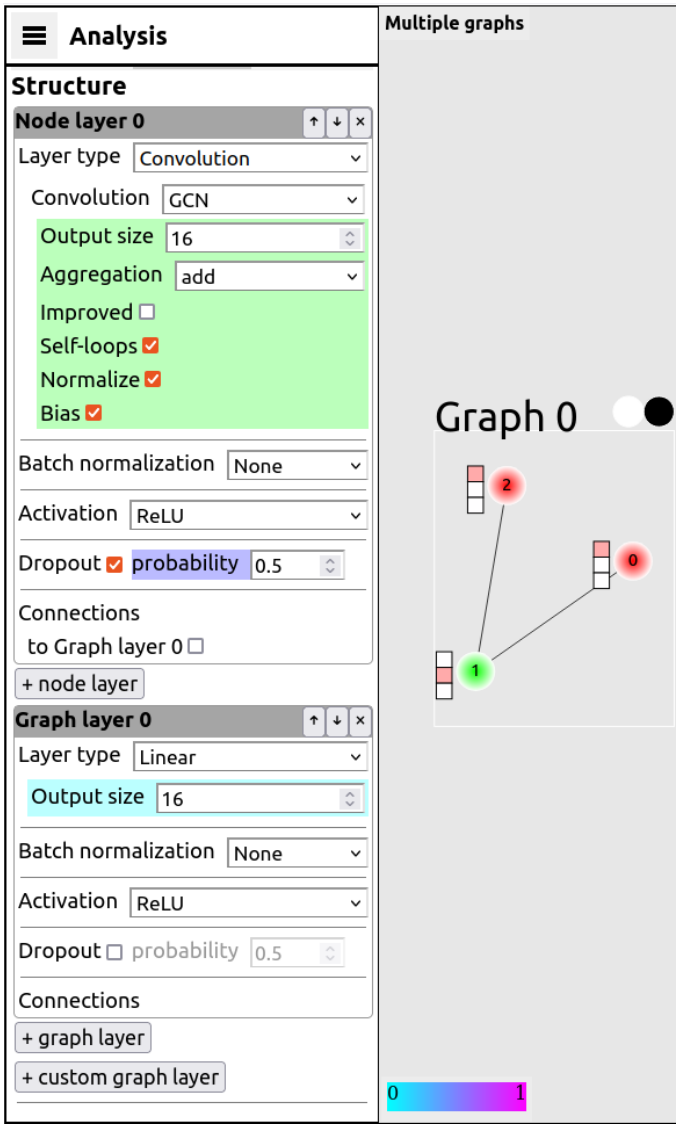

Figure 2: Screenshot of model constructor menu. The model structure in this example starts with a Node layer with a layer of type Convolution, named GCN and output size 16, followed by a batch normalization, ReLU activation unit, and dropout with 0.5 probability. User can add, move, or remove node layers by pressing buttons. The second layer here is a Graph layer of linear type. Below is a button '+ custom graph layer' that adds (currently supported only 1 option) a prototype layer which makes model self-interpretable.

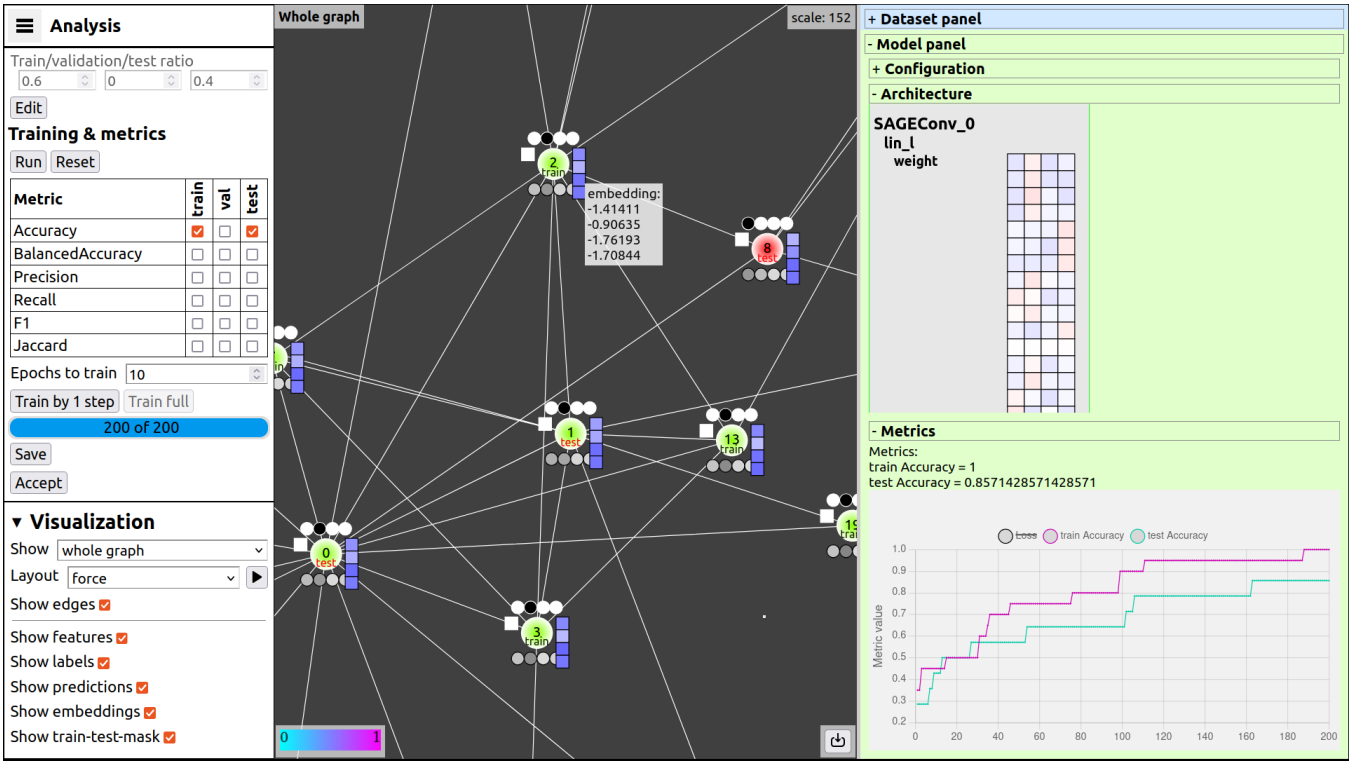

Figure 3: Screenshot of the web interface in analysis mode, suit for introducing beginners to the functionality of GNN models using the karate-club graph as an example. On the left, a menu panel displays model training parameters and visualization control flags for the graph shown in the center. The central section contains graph nodes and edges additional node information: true labels (one of four classes) at the top, normalized prediction vectors (0-1) at the bottom, representation vectors on the right, and a square on the left that reveals feature vectors upon hover (hidden here due to size). Node color indicates its class. On the right, the Model panel displays the model architecture with weights for each layer (numerical values appear on hover over the squares) and a metric plot below, showing their evolution over 200 training epochs.

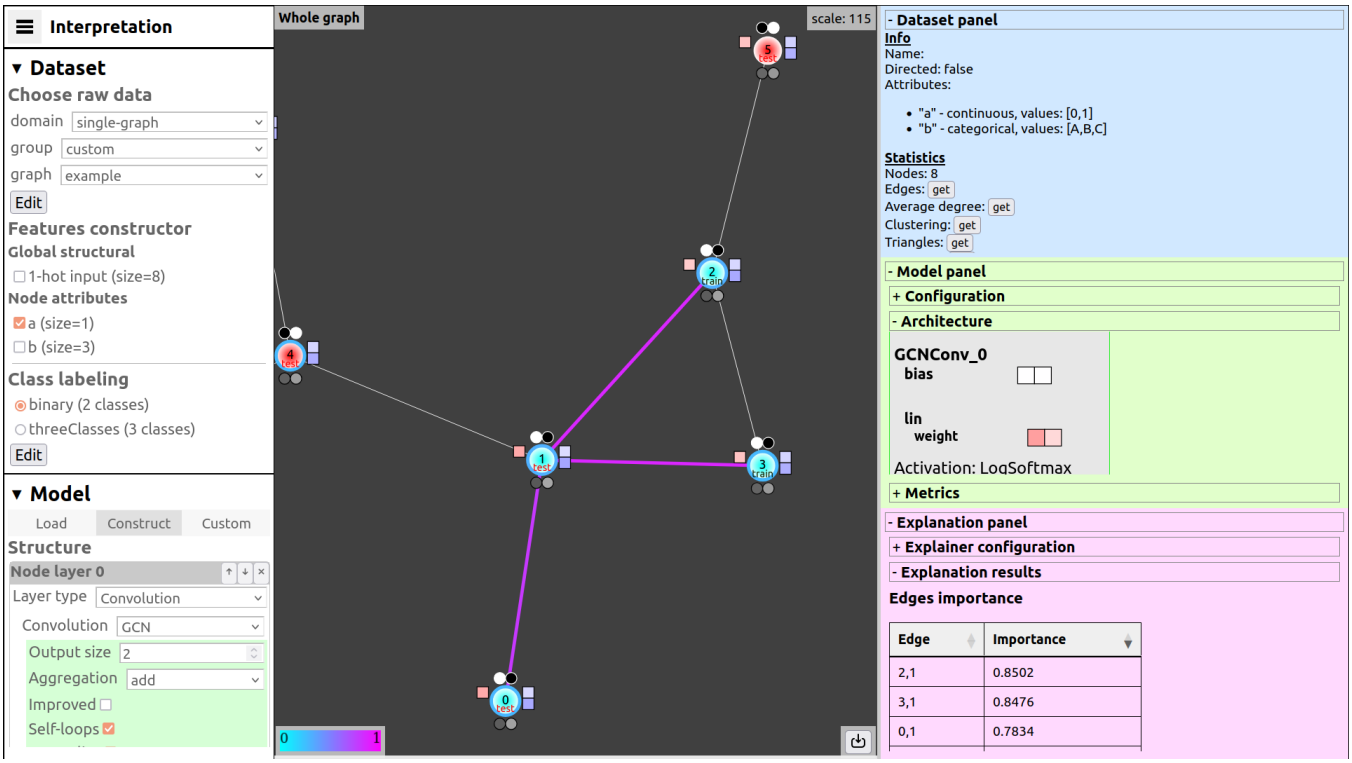

Figure 4: Screenshot of the web interface in interpretation mode. On the left, a menu panel allows selecting the dataset, model (partially visible), and other options (additional elements appear when scrolling down). In the center, a graph visualization is displayed with additional information around the nodes and the results of explaining the classification for node 1, shown as purple edges in its neighborhood. On the right, information panels are arranged from top to bottom: about the dataset, the model, and the interpretation.

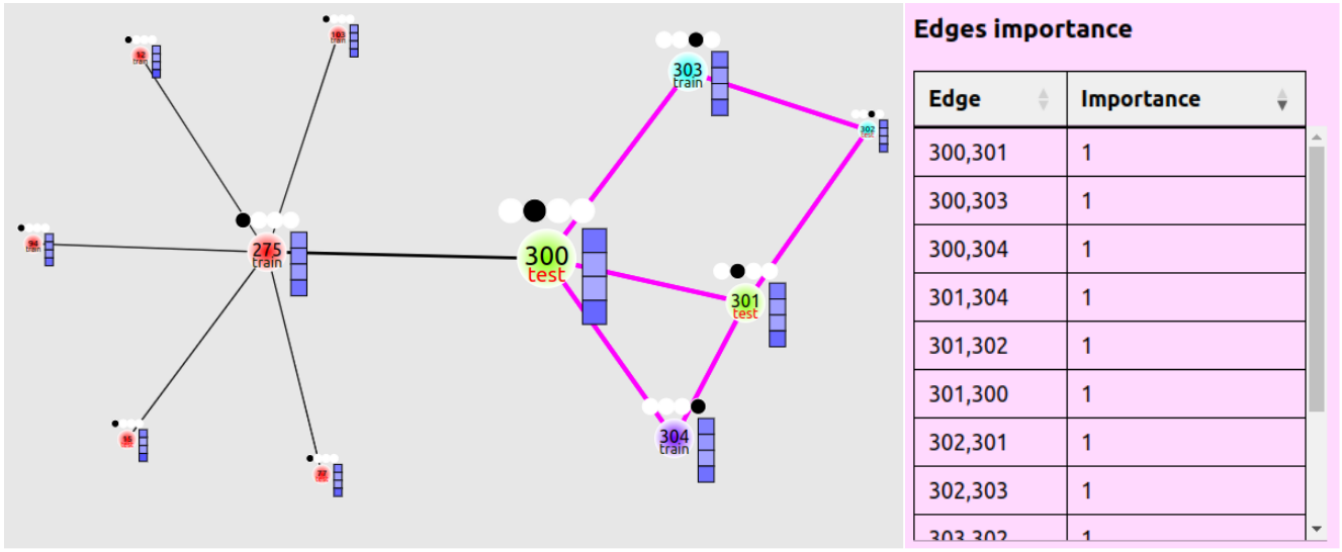

Figure 5: Result of the PGExplainer algorithm on the BAShapes dataset[2]. Interpreting the model's prediction for node 300. The algorithm correctly identifies influential edges in the neighborhood.